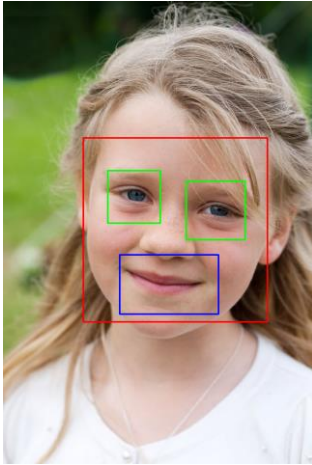


Задание 1.

Необходимо на изображении определить области лица, глаз, губ. Например,



Код для определения области лица (демонстрировался на лекции) :

```
import numpy as np
import cv2
```

```
#load the xml files for face detection into the program
face_cascade=cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive/
#read the image for furthur editing
image = cv2.imread('test_photo.jpg')
```

```
from google.colab.patches import cv2_imshow

#show the original image
#https://upload.wikimedia.org/wikipedia/commons/thumb/7/7a/Google_team_group_photo_with_Vanhanen.jpg/640px-Google_team_group_photo_with_Vanhanen.jpg

cv2_imshow(image)
cv2.waitKey(100)
```

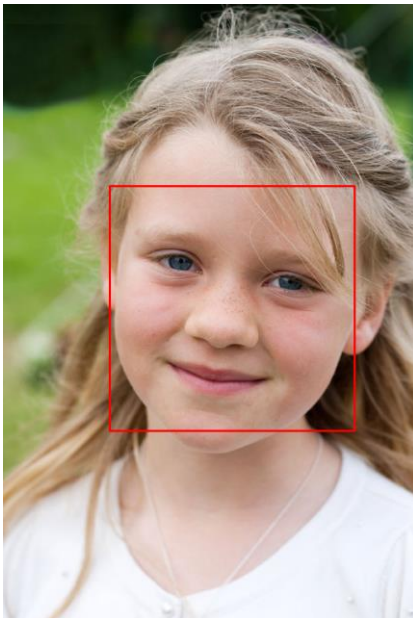


```
#convert the RGB image to gray scale image
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
#identify the face using haar-based classifiers
faces = face_cascade.detectMultiScale(image, 1.4, 4)
```

```
#iteration through the faces array and draw a rectangle
for(x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x+w, y+h), (0, 0, 255), 2)
    roi_gray = gray_image[y:y+h, x:x+w]
    roi_color = image[y:y+h, x:x+w]
```

```
#show the final image after detection
cv2.imshow(image)
cv2.waitKey()
```



Задание 2

Распознавание (идентификация) лиц. Сформировать свою базу.

Код для определения Обамы, Трампа и т.д. (демонстрировался на лекции):

```
!pip install face_recognition
```

```
import os
import cv2
import face_recognition
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
```

```
def load_image(path):
    image = cv2.imread(path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) #1
    return image

def show_image(image):
    plt.imshow(image)
    plt.xticks([])
    plt.yticks([])
    plt.show()
    return
```

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive/templates/

filenames = os.listdir('/content/drive/MyDrive/templates/')
filenames
```

```
mounted at /content/drive
/content/drive/MyDrive/templates
['barack_obama.jpg',
 'bill_gates.jpg',
 'donald_trump.jpg',
 'elon_musk.jpg',
 'steve_jobs.jpg',
 'warren_buffett.jpg']
```

```
images = [load_image(filename) for filename in filenames]
show_image(images[0])
show_image(images[1])
show_image(images[2])
show_image(images[3])
show_image(images[4])
show_image(images[5])
```



```
class Face:
    def __init__(self, bounding_box, cropped_face, name, feature_vector):
        self.bounding_box = bounding_box #1
        self.cropped_face = cropped_face #2
        self.name = name #3
        self.feature_vector = feature_vector #4
```

```

def create_database(filenamees, images):      #1
    faces = []
    for filename, image in tqdm(zip(filenamees, images),
total=len(filenamees)):
        loc = face_recognition.face_locations(image, model='hog')[0]  #2
        vec = face_recognition.face_encodings(image, [loc],
                                                num_jitters=20)[0]      #3

        top, right, bottom, left = loc      #4

        cropped_face = image[top:bottom, left:right]      #5

        face = Face(bounding_box=loc, cropped_face=cropped_face,
                    name=filename.split('.')[0], feature_vector=vec)  #6

        faces.append(face)      #7

    return faces

```

```

faces = create_database(filenamees, images)

```

```

show_image(faces[0].cropped_face)
show_image(faces[1].cropped_face)
show_image(faces[2].cropped_face)
show_image(faces[3].cropped_face)
show_image(faces[4].cropped_face)
show_image(faces[5].cropped_face)

```

```

%cd /content/drive/MyDrive/test/
image_test = load_image('test1.jpg')
show_image(image_test)

```



```

def detect_faces(image_test, faces, threshold=0.6):      #1
    locs_test = face_recognition.face_locations(image_test,
model='hog')      #2
    vecs_test = face_recognition.face_encodings(image_test, locs_test,
                                                num_jitters=1)
#3

    for loc_test, vec_test in zip(locs_test, vecs_test):      #4
        distances = []

```

```

        for face in faces:
            distance = face_recognition.face_distance([vec_test],
                                                    face.feature_vector)

#5
            distances.append(distance)

        if np.min(distances) > threshold: #6
            pred_name = 'unknown'
        else:
            pred_index = np.argmin(distances) #7
            pred_name = faces[pred_index].name

        image_test = draw_bounding_box(image_test, loc_test)
        image_test = draw_name(image_test, loc_test, pred_name)

    return image_test
def draw_bounding_box(image_test, loc_test):
    top, right, bottom, left = loc_test

    line_color = (0, 255, 0)
    line_thickness = 2

    cv2.rectangle(image_test, (left, top), (right, bottom),
                  line_color, line_thickness)

    return image_test

def draw_name(image_test, loc_test, pred_name):
    top, right, bottom, left = loc_test

    font_style = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 1.5
    font_color = (0, 0, 255)
    font_thickness = 3

    text_size, _ = cv2.getTextSize(pred_name, font_style, font_scale,
font_thickness)

    bg_top_left = (left, top-text_size[1])
    bg_bottom_right = (left+text_size[0], top)
    line_color = (0, 255, 0)
    line_thickness = -1

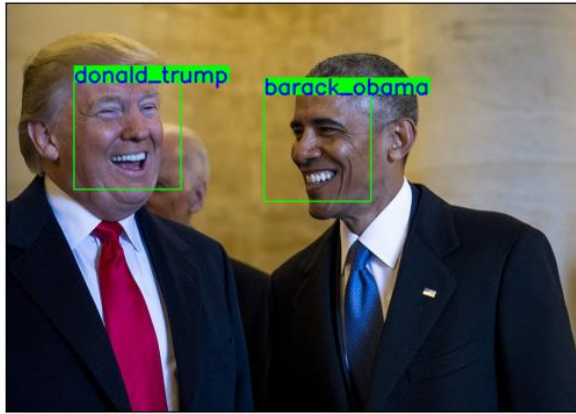
    cv2.rectangle(image_test, bg_top_left, bg_bottom_right,
                  line_color, line_thickness)

    cv2.putText(image_test, pred_name, (left, top), font_style,
font_scale, font_color, font_thickness)

    return image_test

```

```
show_image(detect_faces(image_test, faces))
```



```
show_image(detect_faces(load_image('test2.jpg'), faces))
```



```
show_image(detect_faces(load_image('test3.jpg'), faces))
```

